

HeartbeatSentry

Technical Project Report

Backend API Monitoring Service

Prepared by Roshitha [Gunasekaran]

Github: [roshitha05](#)

Table of Contents

- 1. Project Overview 3
 - 1.1 Project Objectives..... 3
 - 1.2 Scope..... 3
 - 1.2 Public Endpoints..... 3
- 2. Technology Stack 3
- 3. System Architecture 4
 - 3.1 Monitoring Flow 4
- 4. API Design and Implementation 4
 - 4.1 Input Validation and Error Handling..... 5
 - 4.2 Health Classification..... 5
 - 4.3 Example Monitoring Result 5
- 5. Database Design 5
- 6. Monitoring Statistics..... 5
- 7. Automated Testing 6
- 8. Continuous Integration 6
- 9. Deployment 7
- 10. Development Challenges and Solutions 7
- 11. Final Result 7
 - 11.1 Skills Demonstrated..... 7
- 12. Future Improvements 8

1. Project Overview

HeartbeatSentry is a backend API project developed to gain practical experience with REST API development, external HTTP requests, database integration, automated testing, continuous integration and cloud deployment. The service allows web endpoints to be registered and checked for availability. Each check records the HTTP status, response time and health result, while stored results are used to calculate basic uptime statistics.

1.1 Project Objectives

- Build a structured REST API using Python and FastAPI.
- Perform real HTTP health checks against registered web endpoints.
- Store endpoint and monitoring history in a relational database.
- Calculate useful monitoring statistics from saved check results.
- Write automated tests for normal, invalid and failure scenarios.
- Set up continuous integration so tests run automatically on repository changes.
- Deploy the API publicly and connect it to a hosted PostgreSQL database.

1.2 Scope

The current version focuses on the backend monitoring workflow rather than a custom frontend. FastAPI's Swagger interface is used to demonstrate and test the API. The project supports endpoint registration, manual health checks, monitoring history and statistics. Scheduled monitoring, alert notifications, authentication and containerisation are outside the current scope and are listed as possible future improvements.

1.2 Public Endpoints

Live API: <https://heartbeatsentry.onrender.com>

Interactive API documentation: <https://heartbeatsentry.onrender.com/docs>

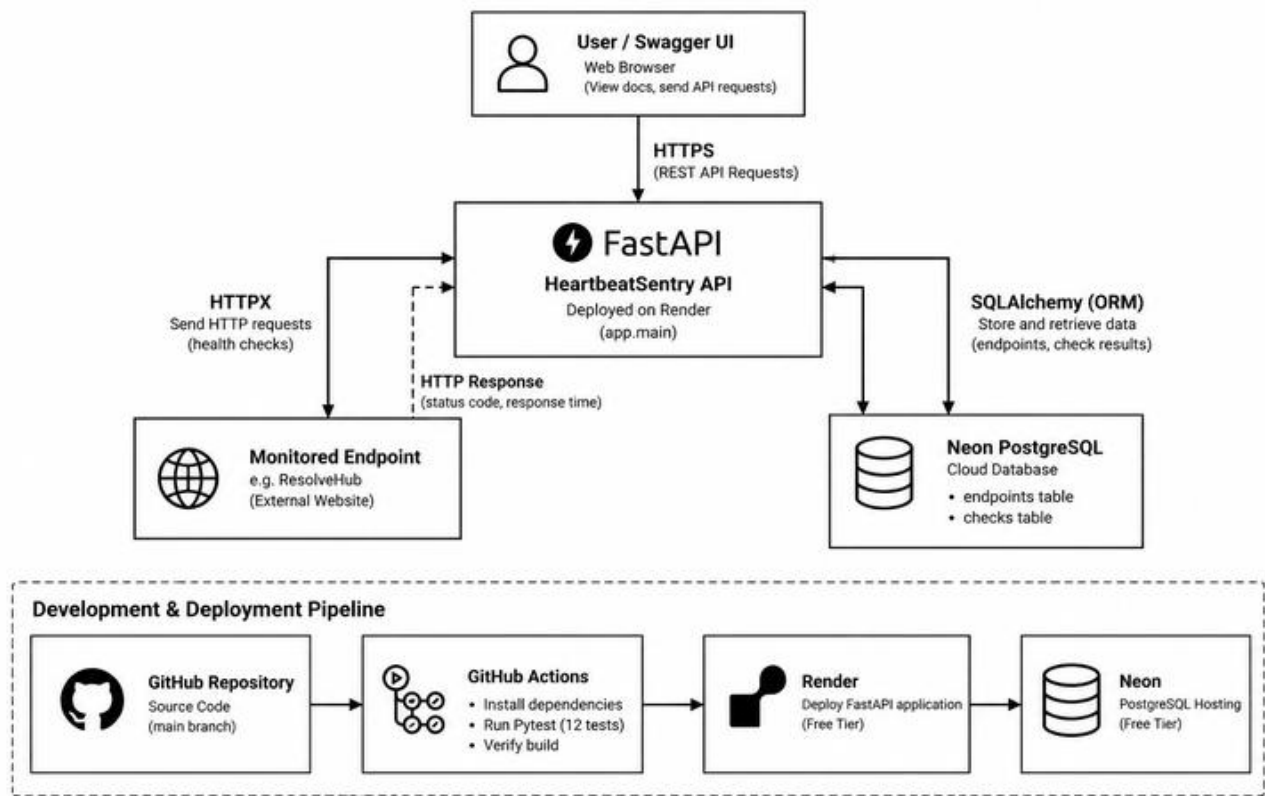
The project uses a free Render instance, so the first request after a period of inactivity may take longer while the service starts.

2. Technology Stack

Technology	Purpose
Python	Core backend language
FastAPI	REST API framework and OpenAPI documentation
Pydantic	Request and URL validation
HTTPX	HTTP requests to monitored endpoints
SQLAlchemy	ORM and database access
SQLite	Default local development database
PostgreSQL / Neon	Hosted production database
Pytest	Automated backend testing
GitHub Actions	Continuous integration
Render	Public FastAPI deployment
Git / GitHub	Version control and repository hosting

3. System Architecture

HeartbeatSentry separates the API, monitoring logic and persistence layer. The deployed FastAPI application runs on Render, performs outbound checks with HTTPX and stores application data in Neon PostgreSQL through SQLAlchemy.



3.1 Monitoring Flow

1. A client registers a URL through the API.
2. HeartbeatSentry stores the endpoint in the database.
3. A health-check request triggers HTTPX to send a real HTTP request to the registered URL.
4. The service records the HTTP status code, response time and any request error.
5. The result is classified as healthy or unhealthy and stored as monitoring history.
6. Statistics are calculated from stored checks and returned through the API.

4. API Design and Implementation

The API is divided into a small set of endpoints that cover service health, endpoint registration, monitoring and statistics. FastAPI also generates interactive OpenAPI documentation automatically at /docs.

Method	Route	Purpose
GET	/	Return basic service information
GET	/health	Check whether HeartbeatSentry is running
POST	/endpoints	Register an endpoint for monitoring
GET	/endpoints	Retrieve registered endpoints
POST	/endpoints/{endpoint_id}/check	Run a health check
GET	/endpoints/{endpoint_id}/checks	Retrieve monitoring history

GET	/endpoints/{endpoint_id}/stats	Retrieve uptime and performance statistics
-----	--------------------------------	--

4.1 Input Validation and Error Handling

Pydantic's `HttpUrl` type validates submitted URLs before they are stored. Duplicate URLs return HTTP 409, invalid URLs are rejected with HTTP 422, and requests for unknown endpoint IDs return HTTP 404. During monitoring, request failures such as connection errors are caught and stored as unhealthy results instead of crashing the API.

4.2 Health Classification

The current monitoring rule treats HTTP 2xx and 3xx responses as healthy. HTTP 4xx and 5xx responses are recorded as unhealthy. Network-level request failures are also recorded as unhealthy with the status code left empty and the error message saved for troubleshooting.

4.3 Example Monitoring Result

```
{
  "endpoint_id": 1,
  "status": "healthy",
  "status_code": 200,
  "response_time_ms": 96.0,
  "error": null
}
```

5. Database Design

SQLAlchemy is used to keep database access consistent between local development and production. SQLite is the default local database, while the deployed application uses PostgreSQL hosted on Neon. The production connection string is supplied through the `DATABASE_URL` environment variable rather than being stored in source code.

Table	Main Fields	Purpose
endpoints	id, name, url, created_at	Stores registered services to monitor
check_results	id, endpoint_id, status, status_code, response_time_ms, error, checked_at	Stores monitoring history

The `check_results` table references endpoints through `endpoint_id`. This allows multiple historical checks to be associated with one monitored endpoint and supports aggregate statistics.

6. Monitoring Statistics

`HeartbeatSentry` calculates statistics from the stored check history for an endpoint. These values provide a simple summary of availability and performance.

- Total number of checks
- Number of successful checks
- Number of failed checks
- Uptime percentage

- Average response time
- Latest health status

Uptime percentage is calculated as successful checks divided by total checks, multiplied by 100. When no checks exist, uptime and average response time return 0.0 and the latest status is null.

7. Automated Testing

The project currently contains 12 automated Pytest tests. Tests use an in-memory SQLite database so the production Neon database and local development data are not modified. External HTTP calls are mocked, which keeps the test suite repeatable and prevents CI results from depending on third-party websites.

Test Area	Examples
Service health	Health route returns the expected response
Endpoint operations	Create and list endpoints
Validation	Invalid URL and duplicate endpoint handling
Error handling	Missing endpoint and connection failure
Monitoring	Healthy 200 and unhealthy 500 responses
Persistence	Health check is stored in history
Statistics	No-check state and mixed healthy/unhealthy calculations

A representative statistics test performs two mocked successful checks and one unsuccessful check, then verifies that the API reports three total checks, two successful checks, one failed check and 66.67% uptime.

8. Continuous Integration

GitHub Actions is configured to run the automated test suite for pushes and pull requests to the main branch. The workflow checks out the repository, sets up Python, installs dependencies from requirements.txt and runs Pytest.

```
Push / Pull Request
  |
  v
Checkout repository
  |
  v
Set up Python
  |
  v
Install dependencies
  |
  v
Run Pytest
  |
  v
Pass / Fail
```

This provides an automated check that the backend still passes its test suite after repository changes. The final project configuration was verified with a successful GitHub Actions run.

9. Deployment

The FastAPI application is publicly deployed on Render using the GitHub repository as its source. The production service starts Uvicorn and reads the Neon PostgreSQL connection string from Render's environment variables.

Component	Deployment
Application	Render Web Service
Database	Neon PostgreSQL
Source repository	GitHub
CI	GitHub Actions
Production configuration	DATABASE_URL environment variable

The deployed service was verified through its public Swagger interface. A real health check was performed against the deployed ResolveHub project, returning HTTP 200 and a healthy status. The resulting check was persisted in Neon and was available through the statistics endpoint.

10. Development Challenges and Solutions

Challenge	Resolution
Using the Neon PostgreSQL connection with SQLAlchemy	Configured the production URL to use the installed psycopg v3 driver by converting <code>postgresql://</code> to <code>postgresql+psycopg://</code> .
Keeping database credentials out of Git	Stored the local connection string in <code>.env</code> , ignored the file in Git, and used environment variables in production.
Testing HTTP monitoring without depending on live websites	Mocked HTTPX responses in Pytest so tests can simulate healthy, unhealthy and connection-failure scenarios.
Separating tests from application data	Overrode the database dependency with an in-memory SQLite database and reset it for each test.
Initial Render deployment attempted to load psycopg2	Committed and deployed the corrected SQLAlchemy driver configuration, after which the service deployed successfully.

11. Final Result

HeartbeatSentry was completed as a working backend portfolio project rather than only a local prototype. The final system provides a documented REST API, performs real endpoint checks, persists monitoring history in PostgreSQL, calculates uptime statistics, includes automated tests, runs continuous integration and is publicly deployed.

11.1 Skills Demonstrated

- Python backend development with FastAPI
- REST API design and HTTP status handling
- Pydantic validation and error handling
- External HTTP integration using HTTPX
- SQLAlchemy ORM and relational database integration
- SQLite and PostgreSQL environment configuration
- Automated testing and mocking with Pytest
- Continuous integration with GitHub Actions
- Environment variable and secret management

- Cloud deployment using Render and Neon
- Git and GitHub version-control workflow

12. Future Improvements

The current scope is intentionally focused on a reliable backend monitoring workflow. If the project is extended, the following improvements would add useful functionality without changing the core design:

- Scheduled automatic health checks and configurable monitoring intervals
- Downtime or recovery notifications
- Endpoint update and deletion operations
- Authentication and per-user monitored endpoints
- Database migrations using Alembic
- Docker containerisation and a more deployment-focused pipeline